

Millicom

Análisis y Diseño De Clientes

Samuel Adonay Alvarado Morales

14 de Abril de 2025



Introducción

Este documento presenta el análisis y diseño de una solución tecnológica para una aplicación web de contenido que recibe aproximadamente 30 millones de visitas mensuales. La mayoría del tráfico (alrededor del 80%) proviene de usuarios anónimos que consumen contenido sin autenticarse, mientras que el 20% restante corresponde a usuarios registrados que acceden a información personalizada una vez inician sesión.

El principal objetivo es proponer una arquitectura que sea capaz de ofrecer tiempos de respuesta menores a 4 segundos, garantizando una experiencia de usuario rápida, consistente y segura, tanto en dispositivos móviles como en computadoras. Para lograrlo, se han considerado elementos clave como el uso de una CDN, balanceo de carga, microservicios escalables, almacenamiento seguro de datos sensibles y un diseño de API RESTful que cumpla con estándares actuales.

A lo largo del documento se explican los distintos componentes propuestos, la infraestructura recomendada, las tecnologías y frameworks sugeridos, así como un diseño detallado del API para el manejo de datos de clientes. Todo esto con el enfoque de construir una solución robusta, escalable y preparada para crecer con las necesidades del sistema.

Requerimiento

Describa la arquitectura recomendada para implementar una aplicación web de contenido con 30 millones de visitas al mes en la cual el 80% del tráfico es anónimo, el 20% restante es de usuarios que al hacer login consultan información personalizada. Tome en consideración que el sitio web puede ser accedido a través de dispositivos móviles y computadoras, se requiere un tiempo de respuesta menor a 4 segundos en la carga de la aplicación. Su respuesta debe considerar lo siguiente :

- Infraestructura para hosting del sitio
- Arquitectura de componentes
- Bases de datos recomendadas
- Frameworks

Basado en su respuesta anterior, tome en consideración que se desea añadir un set de APIs para almacenar y actualizar información de clientes, tomando como identificador un código único asignado. Dentro de la información que se puede actualizar se encuentran:

- Nombre
- Apellido
- Dirección
- Dirección de correo
- Número de tarjeta de crédito
- CVV de la tarjeta de crédito

Se solicita que haga el diseño del API con la que usted implementaría las operaciones de Obtener información del cliente, Actualizar y Eliminar dicha información. Escriba la sintaxis de las peticiones a la API REST tomando en consideración que la respuesta debe expresarse en formato JSON. Indique también el código de respuesta HTTP del resultado de cada petición.



Propuesta de Solución

Objetivos

La solución debe garantizar una experiencia de uso ágil y consistente, asegurando tiempos de respuesta menores a cuatro segundos, incluso bajo condiciones de alta concurrencia. La infraestructura debe contemplar escalabilidad horizontal, balanceo de carga inteligente, alta disponibilidad y desacoplamiento entre componentes funcionales, lo que permitirá una gestión eficiente del tráfico, una evolución modular del sistema y una adecuada tolerancia a fallos.

Se requiere una arquitectura que contemple la distribución de contenido desde bordes de red, el desacoplamiento entre servicios mediante interfaces bien definidas y una separación clara entre las responsabilidades de gestión de identidad, procesamiento de datos y entrega de contenido.

Arquitectura

Se utilizará una Arquitectura de Microservicios con enfoque Event-Driven porque permite construir un sistema altamente escalable, desacoplado y resiliente, ideal para manejar grandes volúmenes de tráfico como el previsto (30 millones de visitas mensuales). Al dividir la aplicación en servicios independientes, cada uno puede evolucionar, desplegarse y escalar por separado según su carga.

Infraestructura para Hosting del Sitio

- **Balanceador de carga (Load Balancer):**
Distribuye las solicitudes entre múltiples instancias del sistema para evitar cuellos de botella y asegurar disponibilidad.
- **CDN (Content Delivery Network):**
Distribuye contenido estático (imágenes, CSS, JS) desde ubicaciones geográficas cercanas a los usuarios. Es clave para reducir la latencia, especialmente para el 80% del tráfico anónimo.

- **Cluster de aplicaciones en contenedores (Kubernetes / App Service):**
Permite el despliegue y escalado automático de los servicios, asegurando rendimiento y resiliencia. Ideal para manejar picos de tráfico.
- **Auto Escalado horizontal:**
Aumenta o disminuye instancias de servicios según la demanda, optimizando costos y garantizando disponibilidad.

Arquitectura de Componentes

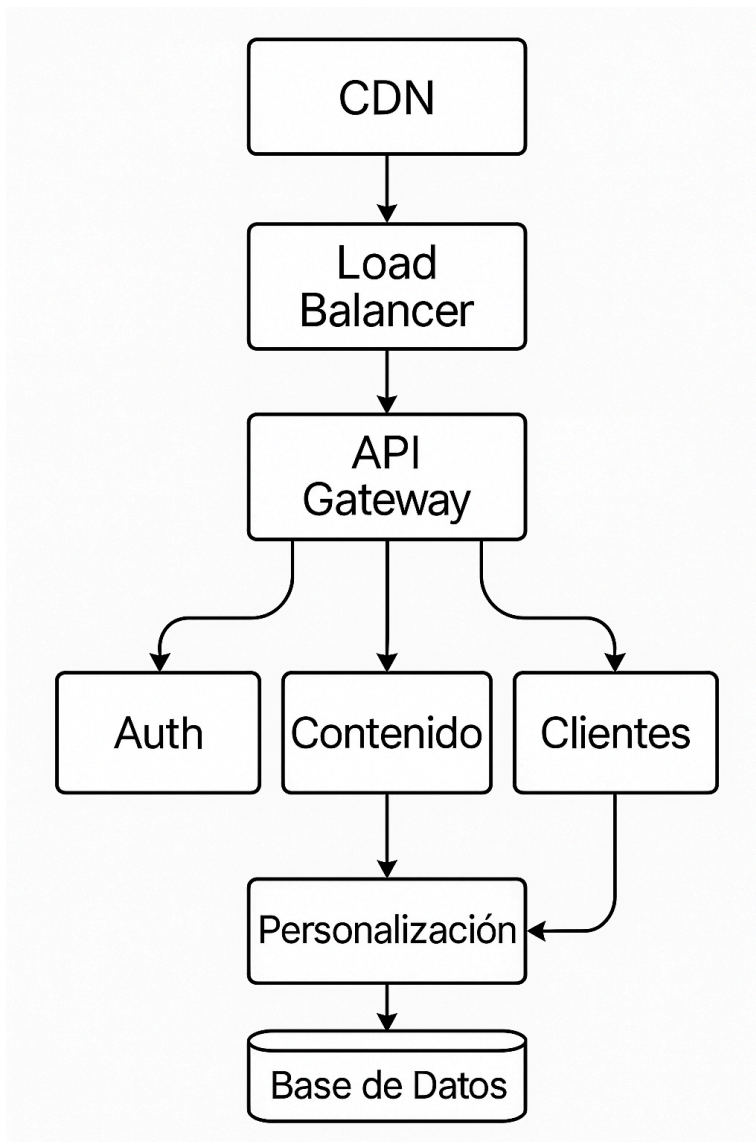
- **Gateway/API Gateway:**
Punto de entrada único. Maneja autenticación, control de acceso, enrutamiento de peticiones y métricas.
- **Servicio de contenido anónimo:**
Maneja la entrega de contenido público. Se prioriza la lectura rápida con soporte de caché.
- **Servicio de autenticación y usuarios:**
Encargado del login, gestión de sesiones (con JWT), y del acceso personalizado para el 20% de tráfico autenticado.
- **Servicio de personalización:**
Retorna contenido específico para el usuario logueado según su perfil y preferencias.
- **Servicio de API de clientes (para la sección 2 del requerimiento):**
Expone endpoints para la gestión de información sensible del cliente de forma segura (nombre, dirección, tarjeta, etc.).
- **Observabilidad (Monitoring y Logging):**
Sistema centralizado de logs, métricas y trazabilidad para asegurar estabilidad.

Frameworks

- **Backend:**
 - Java Spring Boot o Node.js para microservicios.
 - Seguridad con JWT para autenticación.
 - Validaciones robustas para datos sensibles.

Diagrama de Arquitectura de Software

A continuación el diagrama representa una arquitectura escalable para nuestra aplicación web de alto tráfico, organizada en capas. A continuación, explico cada componente de arriba hacia abajo





CDN (Content Delivery Network)

- Es el primer componente que recibe las solicitudes de los usuarios.
- Se encarga de distribuir el contenido de manera eficiente, cacheando contenido estático para mejorar la velocidad y reducir la carga en los servidores.

Load Balancer (Balanceador de carga)

- Recibe las solicitudes desde el CDN.
- Distribuye las solicitudes entrantes entre varios servidores o servicios para asegurar que ninguno se sobrecargue y para mejorar la disponibilidad y escalabilidad.

API Gateway

- Punto central de acceso a los servicios backend.
- Se encarga de recibir las solicitudes de los clientes y enrutar cada solicitud al servicio correspondiente (auth, contenido, clientes).
- Puede manejar seguridad, validación, transformación y monitoreo de las solicitudes.

Servicios backend:

- Auth: Servicio de autenticación, que valida usuarios, gestiona tokens, permisos, etc.
- Contenido: Servicio encargado de manejar el contenido que se muestra o distribuye, puede ser artículos, videos, datos, etc.
- Clientes: Servicio para la gestión de información relacionada con clientes o usuarios.

Personalización (Servicios)

- Este componente recibe datos de los servicios de contenido y clientes para aplicar personalización.
- Aquí se adapta la experiencia o los datos mostrados según el perfil o preferencias del usuario.

- También está conectado con la base de datos para almacenar o recuperar configuraciones y datos relacionados con la personalización.

Base de Datos

- Repositorio central donde se almacenan todos los datos persistentes.
- Incluye datos de usuarios, contenido, preferencias y cualquier otra información necesaria para el sistema.

Flujo general:

1. Un usuario realiza una solicitud que pasa primero por el CDN para optimización.
2. Luego la solicitud se dirige al Load Balancer para distribuir la carga.
3. El Load Balancer envía la solicitud al API Gateway, que decide a qué servicio backend dirigirla (Auth, Contenido o Clientes).
4. Los servicios backend procesan la solicitud y, si es necesario, llaman a Personalización para ajustar la respuesta según el usuario.
5. La Personalización consulta o actualiza datos en la Base de Datos.
6. Finalmente, la respuesta personalizada se devuelve al usuario.

Diseño del API REST para clientes (complemento)

Versionamiento

- Todos los endpoints se versionarán para permitir evolución sin afectar a clientes existentes. Ejemplo:

Ejemplo Ruta

`/api/v1/customers/{codigo_unico}`

Headers esperados

- **Authorization: Bearer <token_JWT>** — Para autenticación y autorización.
- **Content-Type: application/json** — Para especificar formato JSON en el cuerpo de peticiones.
- **X-User-App: <X-User>** –Para validar el usuario por aplicación de servicio
- **X-App-Version:<X-Veersion>** –Para Validar la versión ocupada
- **X-Location:<X-Location-Current>** –Para validar la localización, se envía en formato ISO, para limitar accesos
- **X-Pass-App:<X-Passs>** –Para Autenticar y obtener el token, sólo se utiliza una vez

Manejo estándar de errores (formato JSON)

```
{  
  "code": 400,  
  "message": "El email no tiene formato válido",  
  "errors": [{"message": "error generado, linea tal"}]  
}
```

Códigos HTTP y escenarios adicionales

Acción	Código	Descripción
GET	200 OK	Información retornada correctamente
GET	404 Not Found	Cliente no encontrado
PUT	200 OK	Actualización exitosa
PUT	400 Bad Request	Error de validación
PUT	404 Not Found	Cliente no encontrado

PUT	409 Conflict	Conflicto de actualización (ej. duplicado)
DELETE	204 No Content	Eliminación exitosa
DELETE	404 Not Found	Cliente no encontrado
Cualquier	401 Unauthorized	Token inválido o faltante
Cualquier	403 Forbidden	No autorizado para la operación

Consideraciones de seguridad API

- **Encriptación de CVV:** Solo se valida en tiempo real, nunca se guarda ni se expone.
- **Número de tarjeta enmascarado:** Solo se retorna el último bloque visible (ej. ****
**** **** **1234**).
- **Rate limiting:** Para proteger el API de abusos o ataques de fuerza bruta.
- **Auditoría:** Registro de cambios y accesos a información sensible para fines de seguridad y cumplimiento.

Seguridad adicional

- **Cumplimiento PCI-DSS:** Para el manejo de datos de tarjetas de crédito, cumpliendo estándares para procesamiento, almacenamiento y transmisión segura.
- **Validación y sanitización:** En todas las entradas para evitar inyección de código o datos maliciosos.
- **Autorización basada en roles:** Control estricto sobre qué usuarios o sistemas pueden consultar o modificar datos.
- **Monitoreo de seguridad:** Detección y alerta ante accesos no autorizados o patrones anómalos.



Experiencia y rendimiento

- **Pruebas de carga y rendimiento:** Para asegurar que el tiempo de respuesta sea menor a 4 segundos, incluso en picos de tráfico.
- **Optimización móvil:** Uso de diseño responsivo, lazy loading, compresión de recursos, y optimización de imágenes.
- **CDN con caché inteligente:** Para acelerar la entrega y disminuir la carga en backend.



Conclusiones

Después de analizar los requerimientos y las características del proyecto, propongo una arquitectura moderna basada en microservicios que permitirá escalar fácilmente y mantener una buena experiencia de usuario, incluso con un volumen alto de tráfico como el que se espera (30 millones de visitas al mes).

La distribución del contenido a través de una CDN y el uso de balanceadores de carga ayudan a mantener tiempos de respuesta bajos, lo cual es clave especialmente para el 80% del tráfico que es anónimo. Además, separar los servicios por responsabilidades (autenticación, contenido, clientes, personalización) facilita el mantenimiento, mejora la seguridad y permite que cada componente escale según lo necesite.

Para los usuarios autenticados y el manejo de datos sensibles como información personal y tarjetas de crédito, tomé en cuenta las mejores prácticas de seguridad, cumpliendo con normativas como PCI-DSS. El diseño del API REST es simple, seguro y claro, pensado para integrarse fácilmente con otras aplicaciones en el futuro.

